

Data Structures

Lecture 15: Quick Sort

Soobin Um

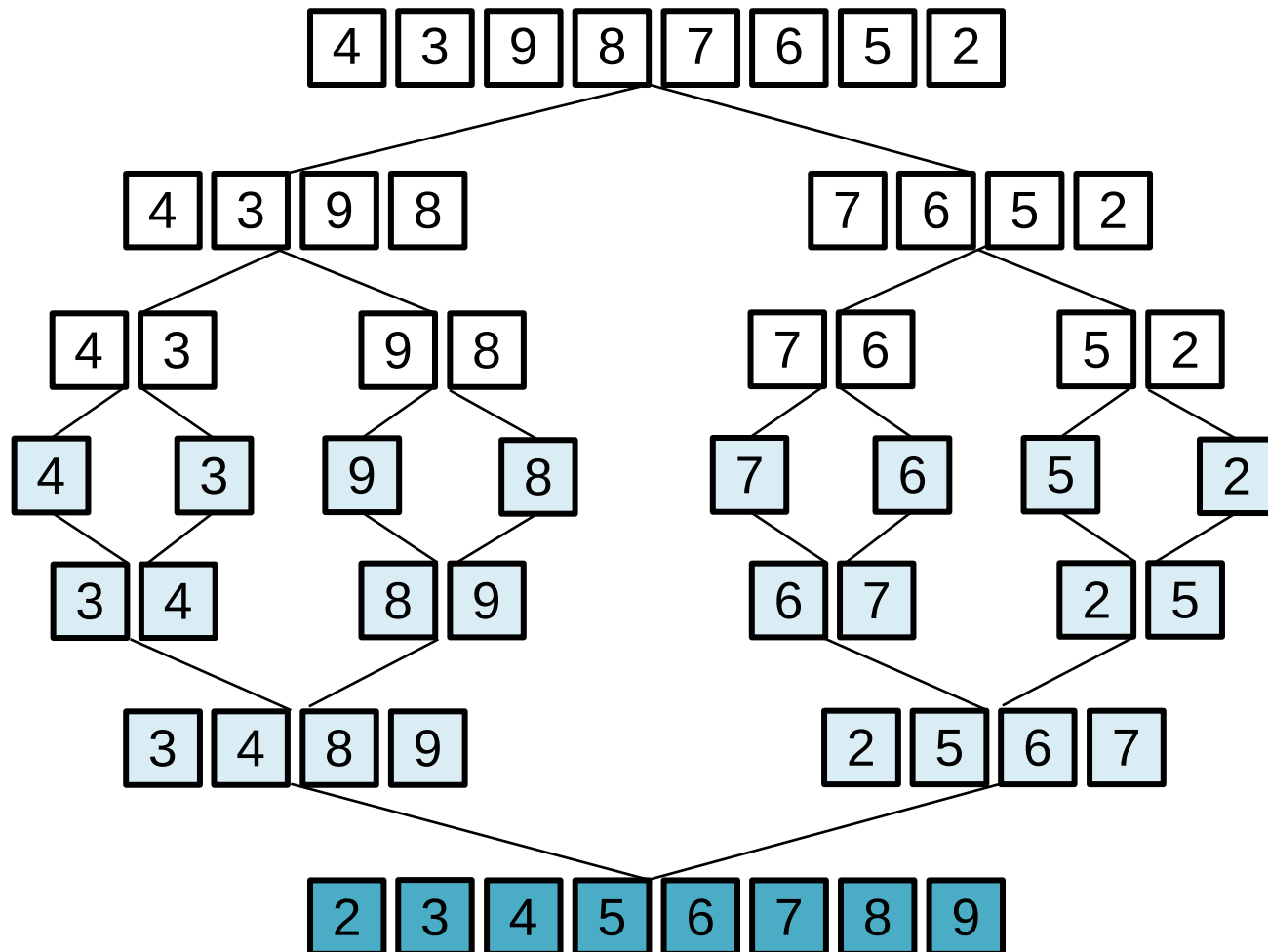
May 6, 2026

Recap: Divide and Conquer (분할 정복 전략)

1. 큰 문제를 여러 개의 작은 하위 문제로 나눔 (divide)
2. 각 하위 문제를 개별적으로 해결 (conquer)
3. 하위 문제들의 해결 결과를 결합하여 (merge) 원래 문제를 해결



Recap: Merge Sort



Quick Sort

Quick Sort

1. Divide:
2. Conquer:
3. Merge:

Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer:
3. Merge:

Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer: 퀵 정렬을 재귀적으로 호출함으로써, 각 부분 배열을 정렬
3. Merge:

Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer: 퀵 정렬을 재귀적으로 호출함으로써, 각 부분 배열을 정렬
3. Merge: 별도의 병합 작업 불필요

Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer: 퀵 정렬을 재귀적으로 호출함으로써, 각 부분 배열을 정렬
3. Merge: 별도의 병합 작업 불필요

QuickSort ($A[], p, r$) $\triangleright$ A : 배열, p : 시작 위치, r : 끝 위치

```
1  if ( $p < r$ ) then  
2       $q = \text{Partition}(A, p, r)$   
3      QuickSort( $A, p, q-1$ )  
4      QuickSort( $A, q+1, r$ )
```

Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer: 퀵 정렬을 재귀적으로 호출함으로써, 각 부분 배열을 정렬
3. Merge: 별도의 병합 작업 불필요

QuickSort ($A[], p, r$) $\triangleright$ A : 배열, p : 시작 위치, r : 끝 위치

```
1  if ( $p < r$ ) then
2     $q = \text{Partition}(A, p, r)$ 
3    QuickSort( $A, p, q-1$ )
4    QuickSort( $A, q+1, r$ )
```

$A[p \dots r]$ 의 원소들을 $A[q]$ 을 기준으로 양쪽으로 재배치 한 뒤,
 $A[q]$ 이 자리한 위치를 반환

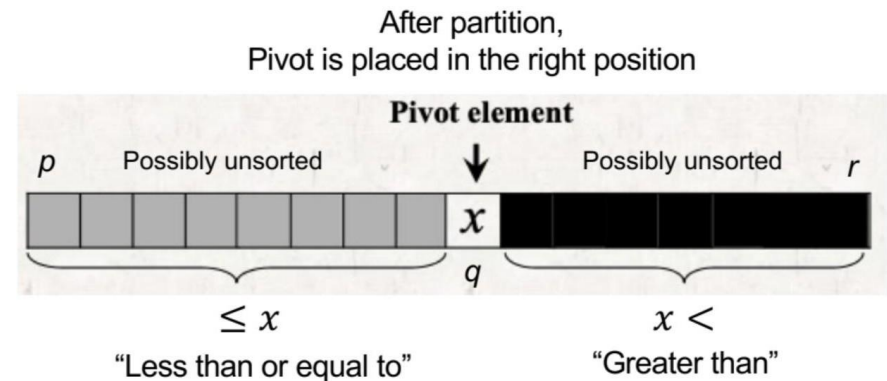
Quick Sort

1. Divide: 입력 배열을 두 개의 부분 배열로 분할
 - 왼쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 작거나 같음
 - 오른쪽 부분 배열의 모든 원소는 **피벗(Pivot)**보다 큼
2. Conquer: 퀵 정렬을 재귀적으로 호출함으로써, 각 부분 배열을 정렬
3. Merge: 별도의 병합 작업 불필요

QuickSort ($A[], p, r$) $\triangleright$ A : 배열, p : 시작 위치, r : 끝 위치

```
1  if ( $p < r$ ) then
2     $q = \text{Partition}(A, p, r)$ 
3    QuickSort( $A, p, q-1$ )
4    QuickSort( $A, q+1, r$ )
```

$A[p \dots q]$ 의 원소들을 $A[q]$ 을 기준으로 양쪽으로 재배치 한 뒤,
 $A[q]$ 이 자리한 위치를 반환



Partition (분할)

배열을 피벗 기준으로 두 영역으로 나눔

- 왼쪽 배열(영역)엔 **피벗 이하**, 오른쪽 배열엔 **피벗 이상의 수**들이 위치하도록 재배치
- 분할이 완료되면, 피벗이 최종적으로 놓이게 되는 **인덱스(분할 경계)를 반환**

분할 과정

왼쪽 배열의 마지막을 가리키는 인덱스 i 와 오른쪽 배열의 마지막을 가리키는 j 를 조정

Partition ($A[], p, r$)

1 $x \leftarrow A[r]$ ▷ 피벗

2 $i \leftarrow p - 1$

3 **for** $j \leftarrow p$ **to** $r - 1$

4 **if** ($A[j] \leq x$) **then** $A[++i] \leftrightarrow A[j]$

5 $A[i+1] \leftrightarrow A[r]$

6 **return** $i + 1$

분할 과정

왼쪽 배열의 마지막을 가리키는 인덱스 i 와 오른쪽 배열의 마지막을 가리키는 j 를 조정

Partition ($A[], p, r$)

1 $x \leftarrow A[r]$ ▷ 피벗

2 $i \leftarrow p - 1$

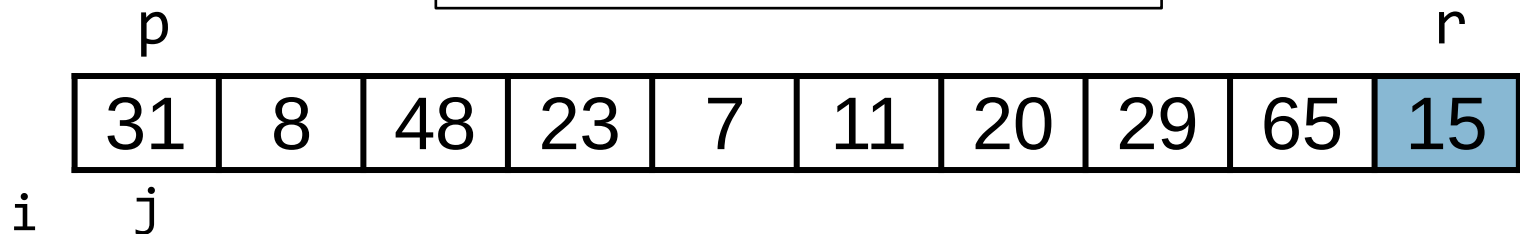
3 **for** $j \leftarrow p$ **to** $r - 1$

4 **if** ($A[j] \leq x$) **then** $A[++i] \leftrightarrow A[j]$

5 $A[i+1] \leftrightarrow A[r]$

6 **return** $i + 1$

(분할 전)

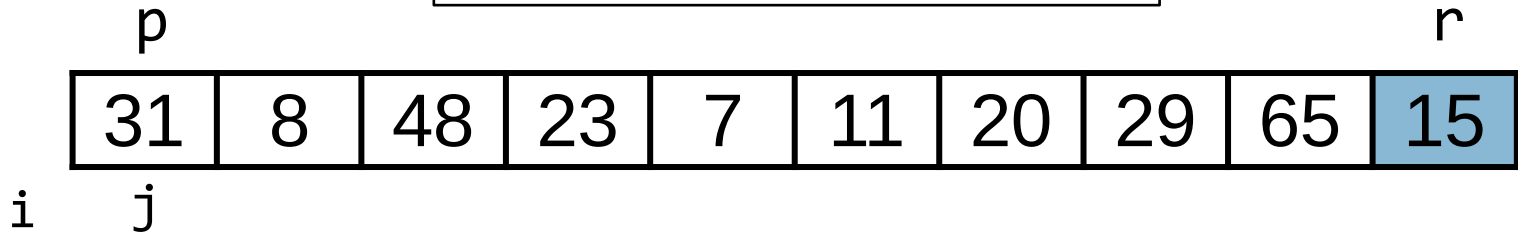


분할 과정

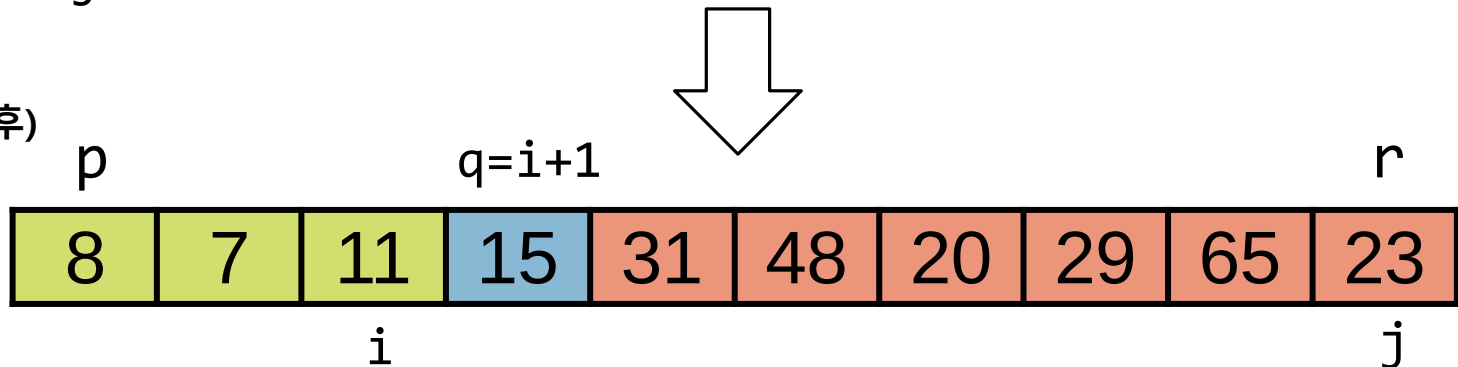
왼쪽 배열의 마지막을 가리키는 인덱스 i 와 오른쪽 배열의 마지막을 가리키는 j 를 조정

```
Partition ( $A[], p, r$ )  
1   $x \leftarrow A[r]$  ▷ 피벗  
2   $i \leftarrow p - 1$   
3  for  $j \leftarrow p$  to  $r - 1$   
4      if ( $A[j] \leq x$ ) then  $A[++i] \leftrightarrow A[j]$   
5   $A[i+1] \leftrightarrow A[r]$   
6  return  $i + 1$ 
```

(분할 전)



(분할 후)



Quick Sort

31	8	48	23	7	11	20	29	65	15
----	---	----	----	---	----	----	----	----	----

Quick Sort

31	8	48	23	7	11	20	29	65	15
----	---	----	----	---	----	----	----	----	----

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

Quick Sort

31	8	48	23	7	11	20	29	65	15
----	---	----	----	---	----	----	----	----	----

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

Pivot 선택

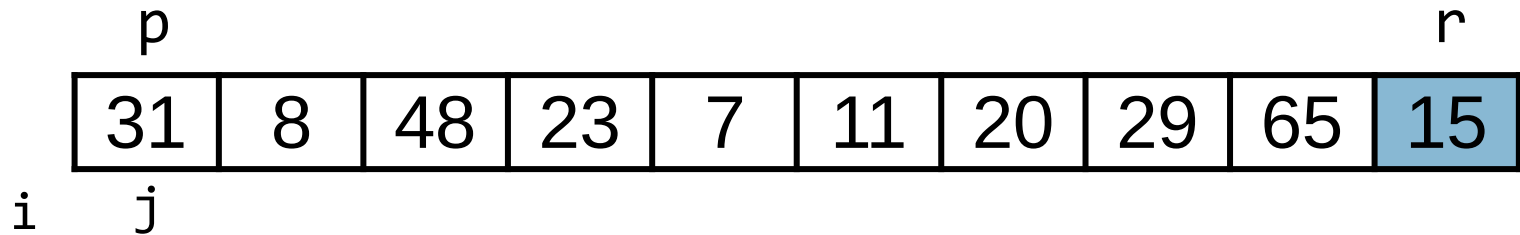
									Pivot
31	8	48	23	7	11	20	29	65	15

배열의 가장 마지막 원소를 피벗(Pivot)으로 사용(Lomuto Partition 방식)

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r]; ②  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r); ①  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

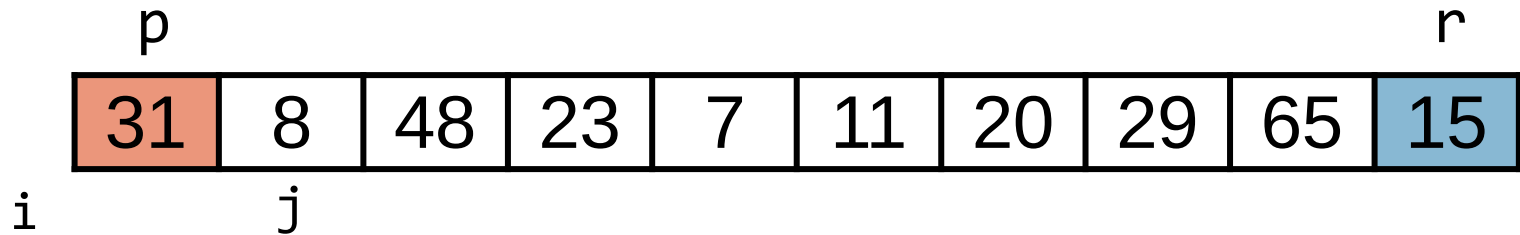
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1; ①  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

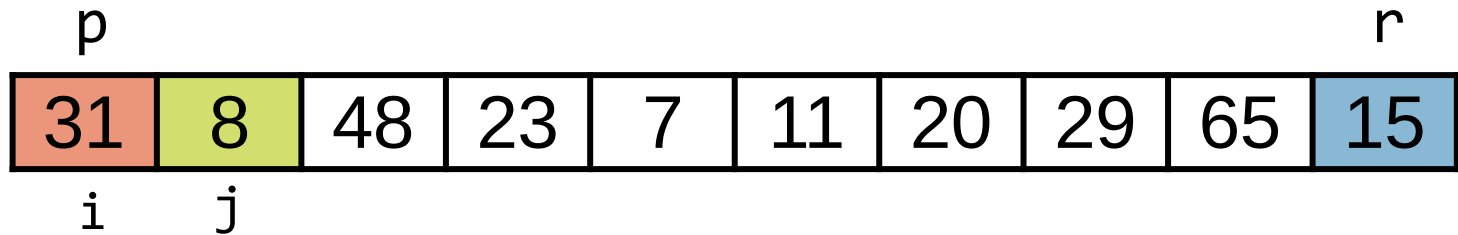
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

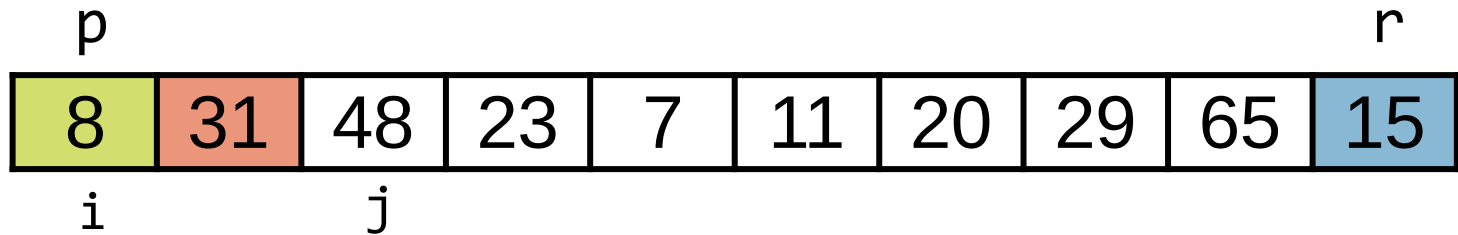
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

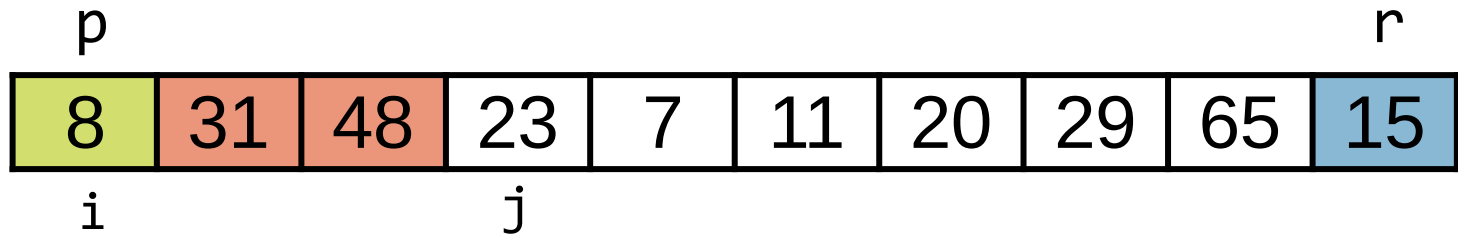
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j); ①  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

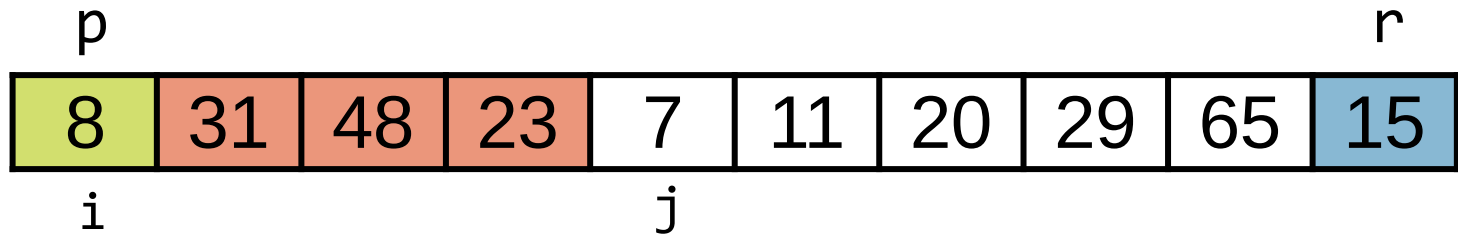
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

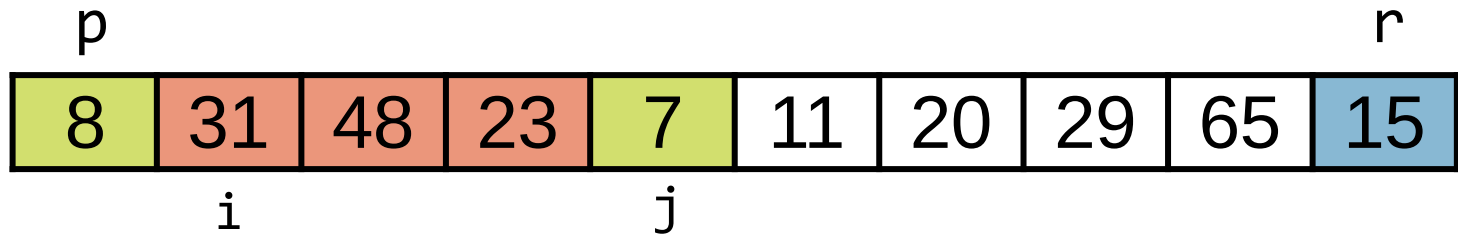
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

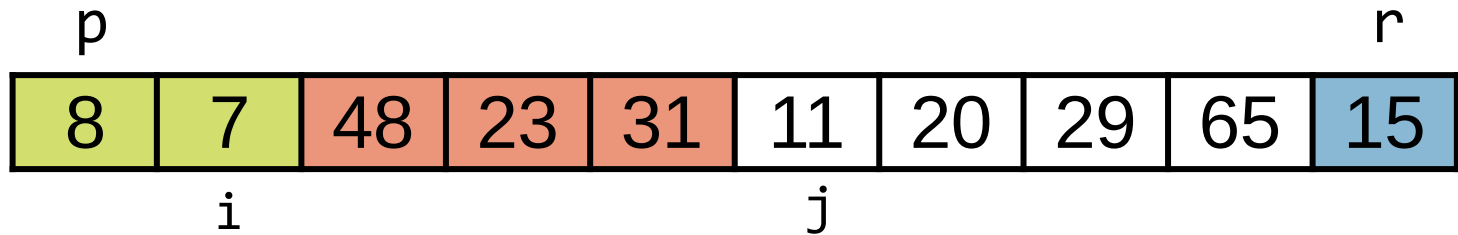
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

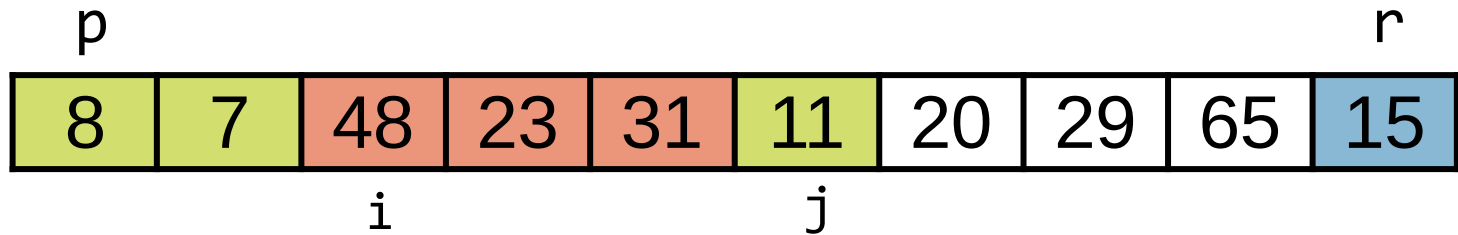
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j); ①  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

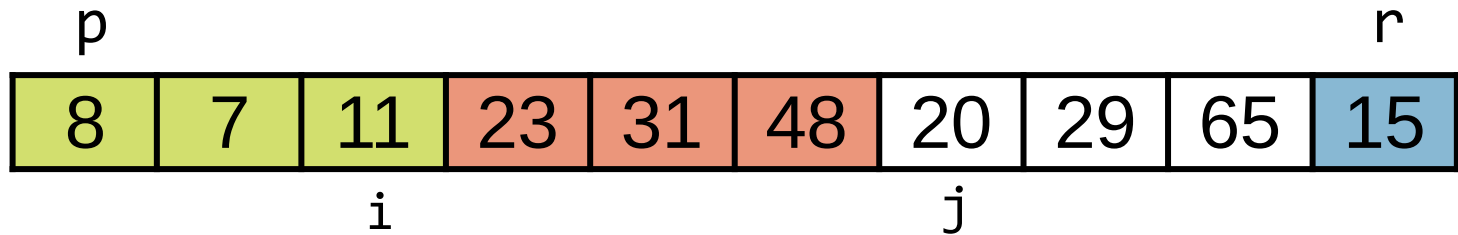
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

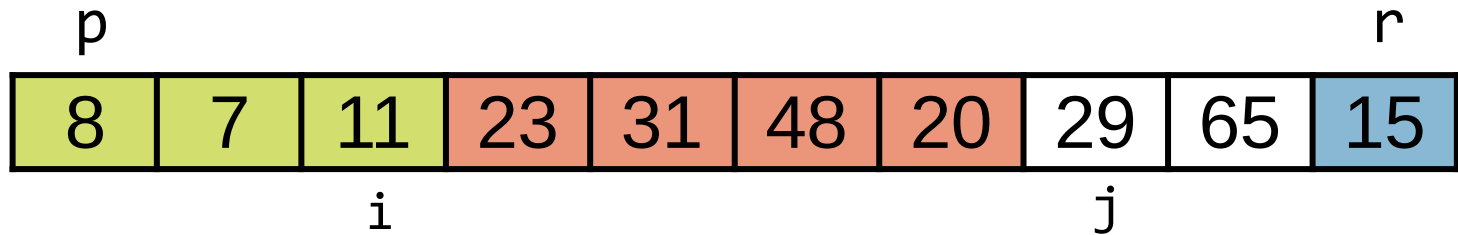
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j); ①  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

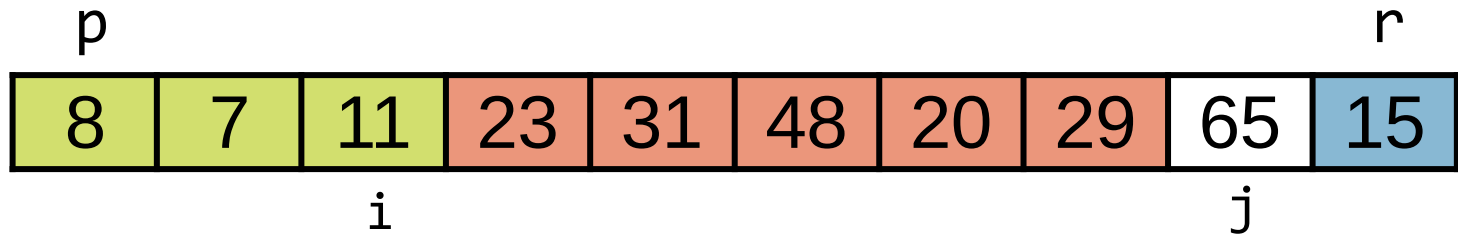
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

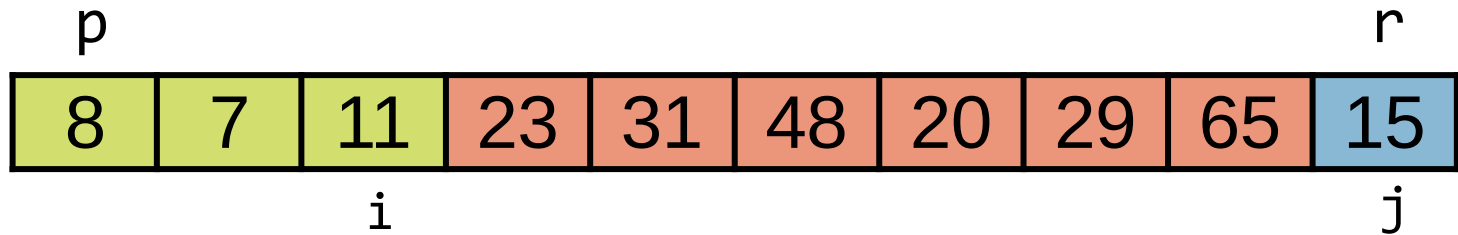
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

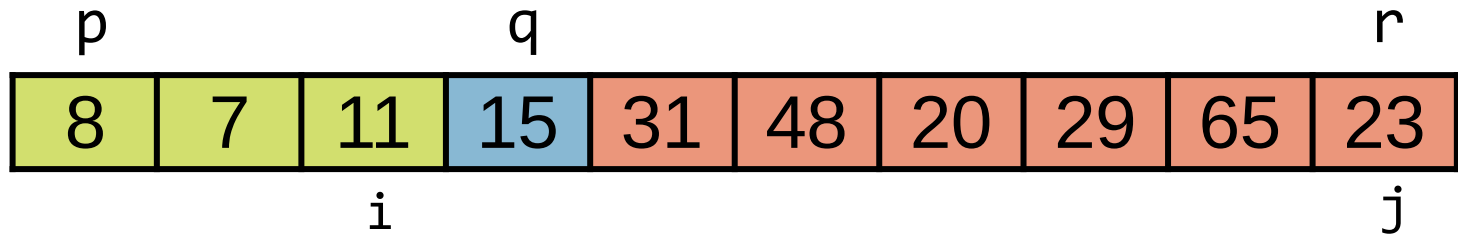
분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

분할 지점 탐색 (Pivot: 15)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ①  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r); ② // 피벗 이동  
    return i + 1;      ③ // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r); ④  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

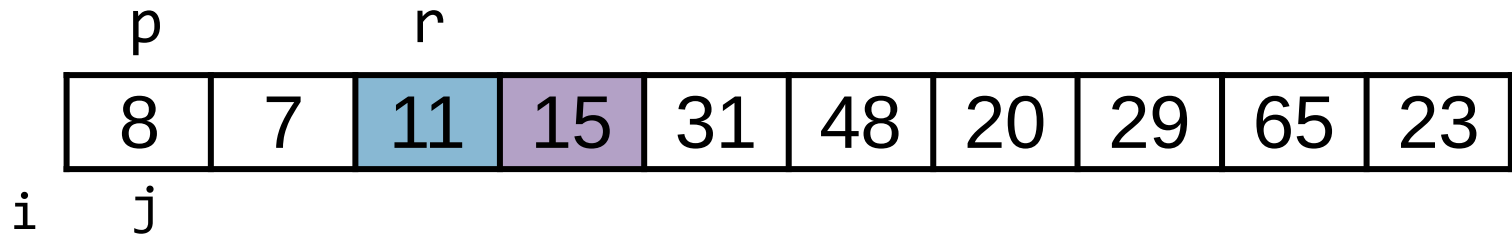
왼쪽 부분 배열 대상 정렬 (Pivot: 11)

p		r	q						
8	7	11	15	31	48	20	29	65	23

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r]; ②  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1); ①  
        quickSort(A, q + 1, r);  
    }  
}
```

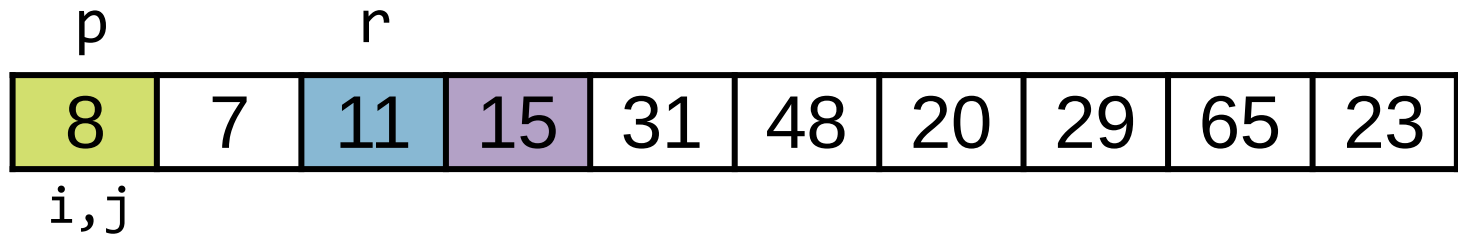
왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1; ①  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

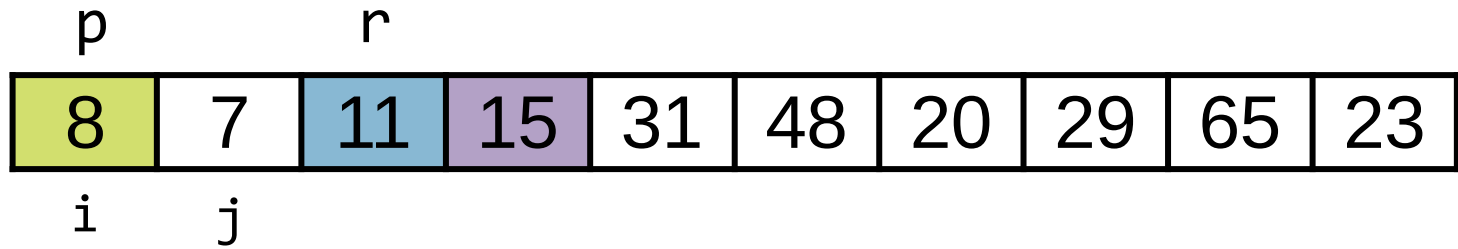
왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++; ①  
            swap(A, i, j); ② 자리 이동 없음  
        }  
    }  
    swap(A, i + 1, r); // 피벗 이동  
    return i + 1;      // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

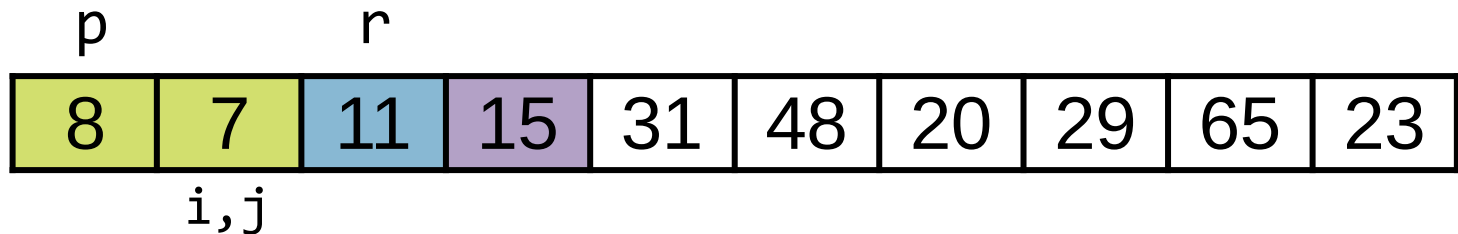
왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

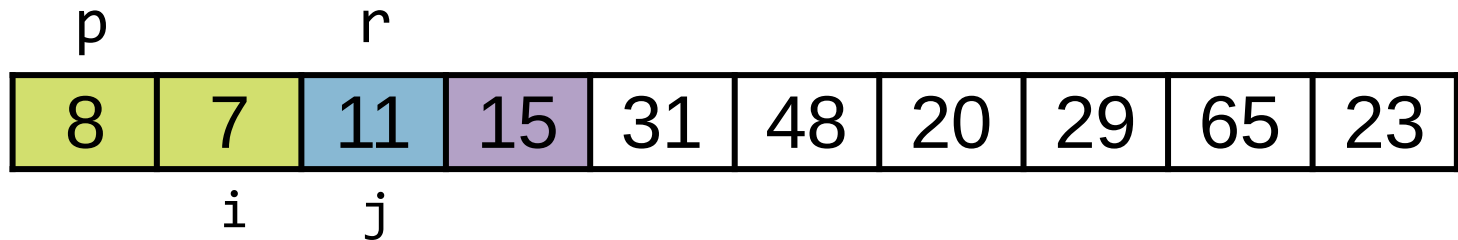
왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++; ①  
            swap(A, i, j); ② 자리 이동 없음  
        }  
    }  
    swap(A, i + 1, r); // 피벗 이동  
    return i + 1;     // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

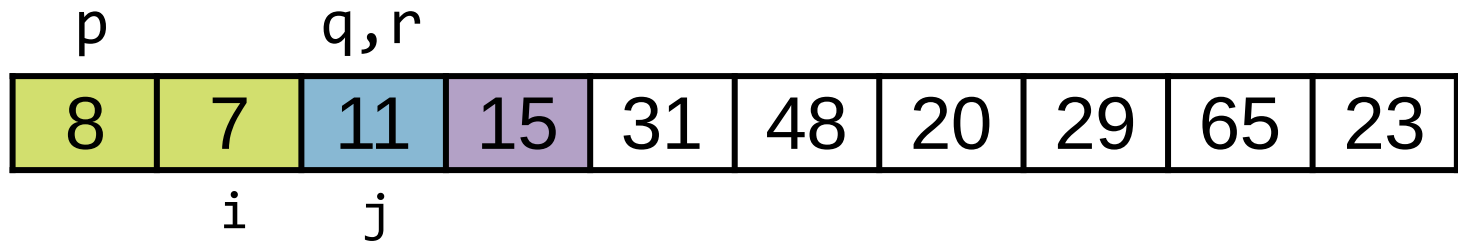
왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

왼쪽 부분 배열 대상 정렬 (Pivot: 11)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ①  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r); ② // 피벗 이동  
    return i + 1; ③ // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r); ④  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

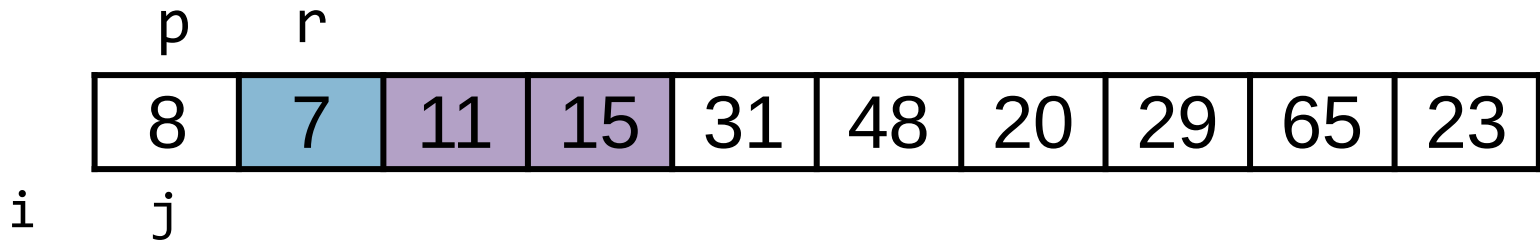
왼쪽 부분 배열 대상 정렬 (Pivot: 7)

	p		r						
8	7	11	15	31	48	20	29	65	23

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r]; ②  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r); ①  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

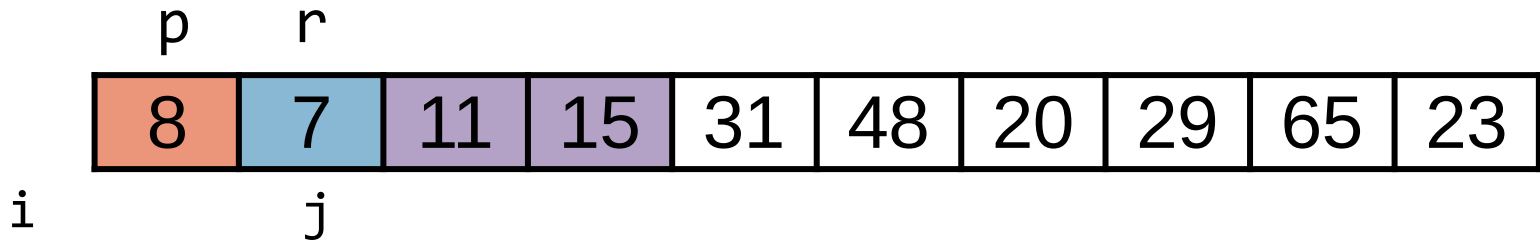
왼쪽 부분 배열 대상 정렬 (Pivot: 7)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1; ①  
    for (int j = p; j < r; j++) { ②  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

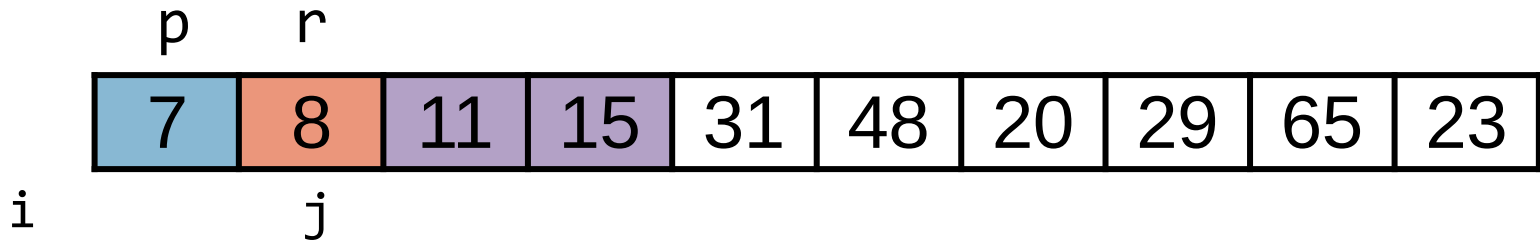
왼쪽 부분 배열 대상 정렬 (Pivot: 7)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

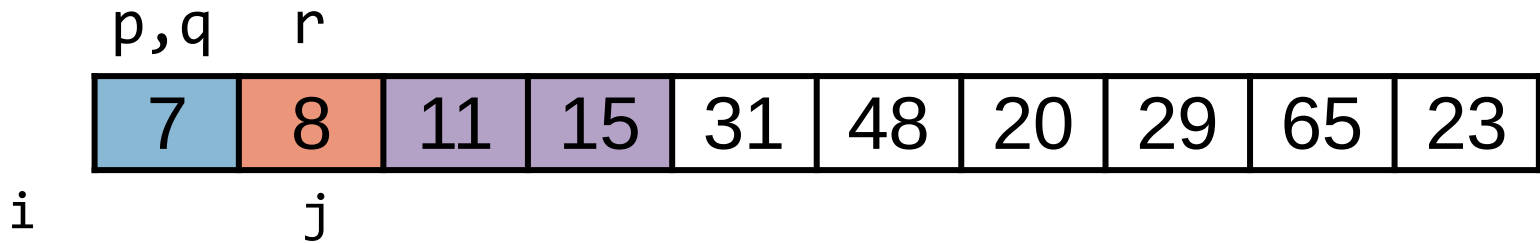
왼쪽 부분 배열 대상 정렬 (Pivot: 7)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) { ①  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r); ② // 피벗 이동  
    return i + 1;      // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

왼쪽 부분 배열 대상 정렬 (Pivot: 7)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r); // 피벗 이동  
    return i + 1; ③ // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r); ④  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

왼쪽 부분 배열 대상 정렬

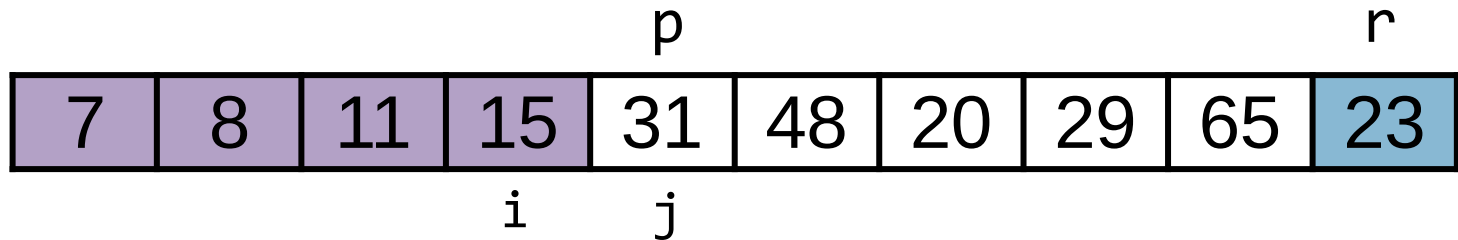
r p

7	8	11	15	31	48	20	29	65	23
---	---	----	----	----	----	----	----	----	----

```
int partition(int[] A, int p, int r) {
    int pivot = A[r];
    int i = p - 1;
    for (int j = p; j < r; j++) {
        if (A[j] <= pivot) {
            i++;
            swap(A, i, j);
        }
    }
    swap(A, i + 1, r);    // 피벗 이동
    return i + 1;        // 분할 지점 반환
}
```

```
void quickSort(int[] A, int p, int r) {
    if (p < r) { ②
        int q = partition(A, p, r);
        quickSort(A, p, q - 1); ①
        quickSort(A, q + 1, r);
    }
}
```

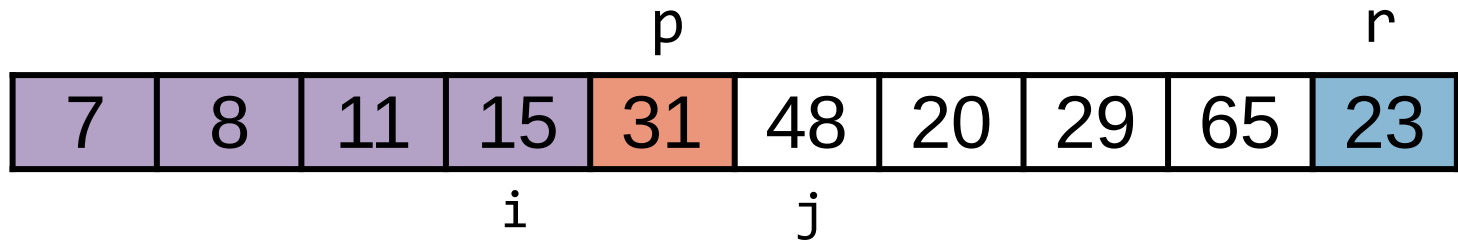
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

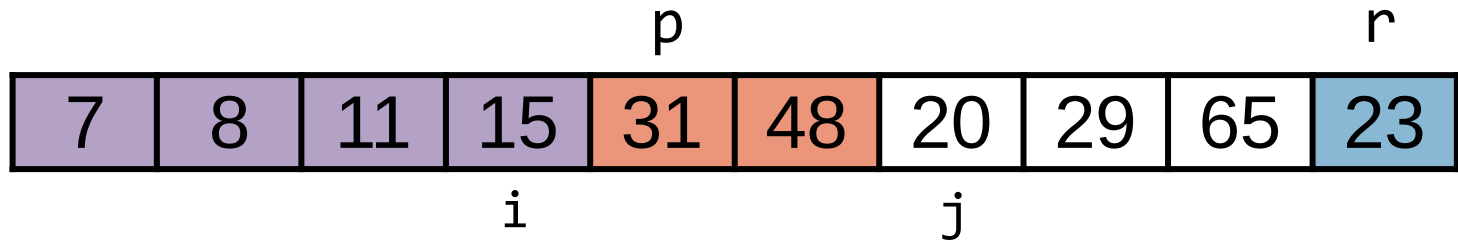
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

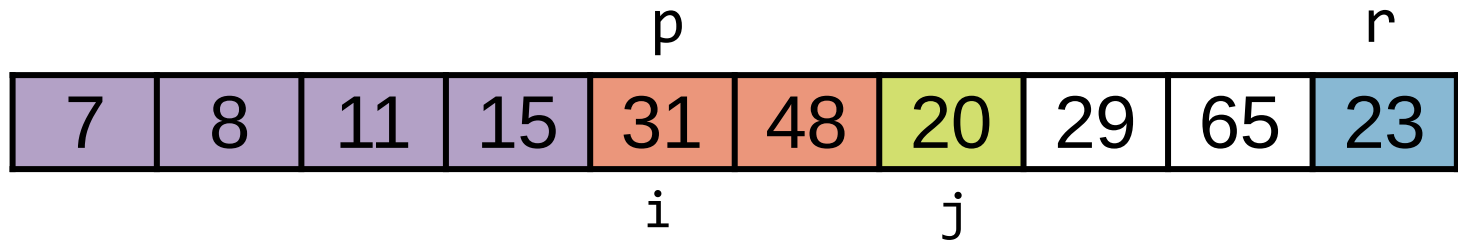
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

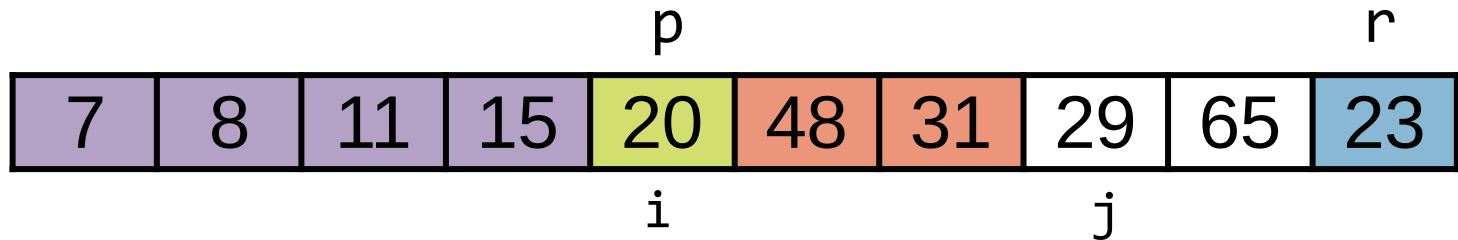
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

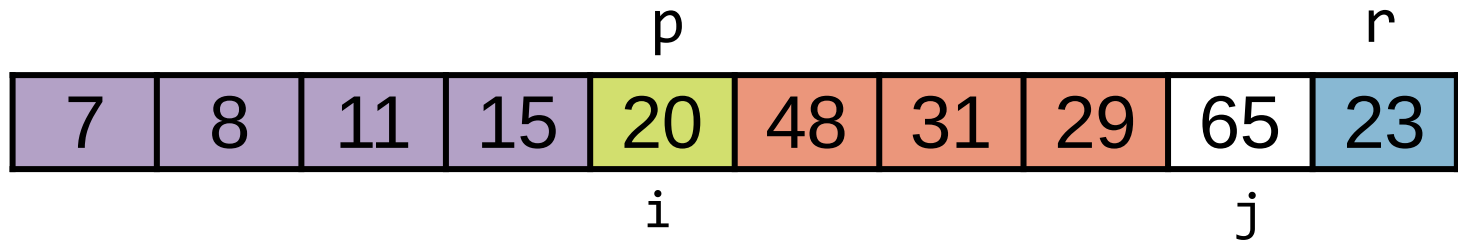
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

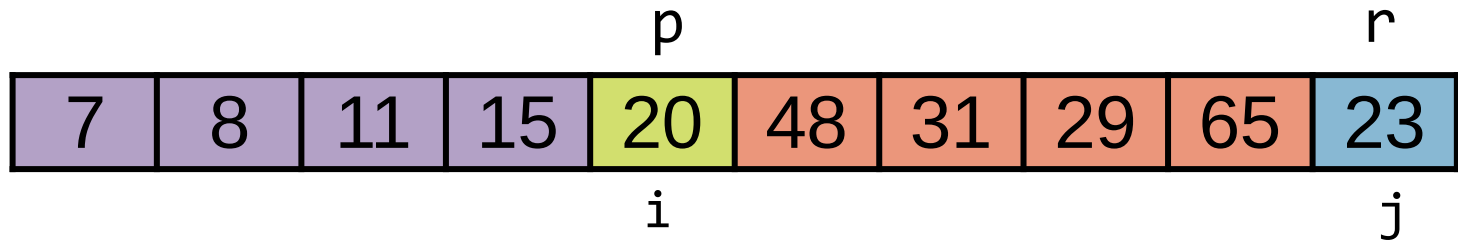
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

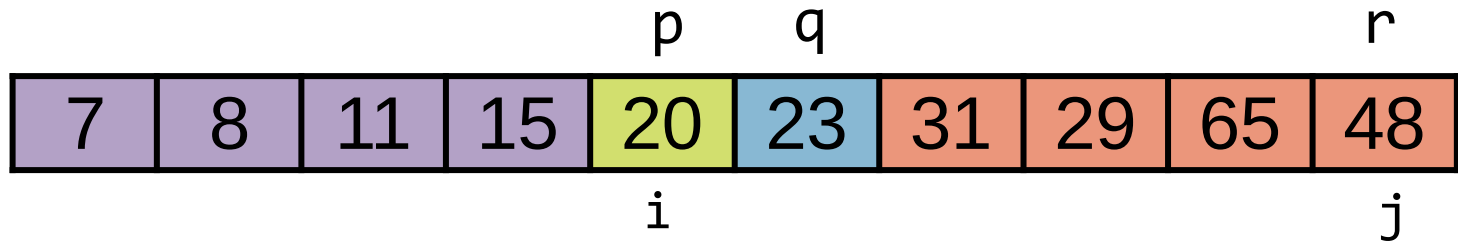
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

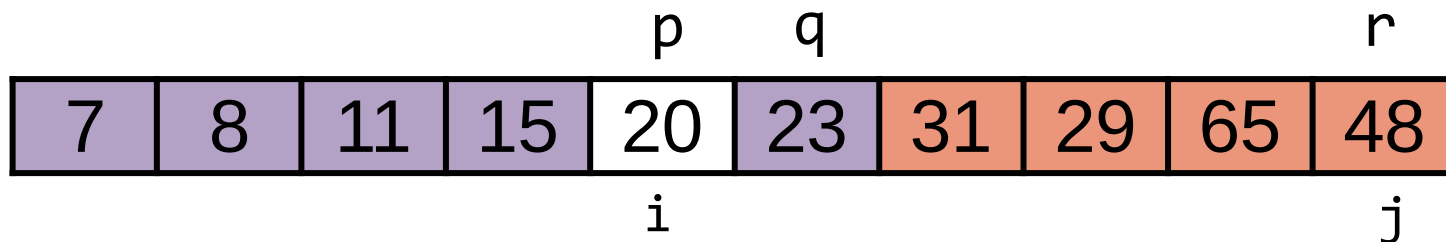
오른쪽 부분 배열 대상 정렬 (Pivot: 23)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);  
    return i + 1;  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬

p, r

7	8	11	15	20	23	31	29	65	48
---	---	----	----	----	----	----	----	----	----

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬

						p			r
7	8	11	15	20	23	31	29	65	48

```
int partition(int[] A, int p, int r) {
    int pivot = A[r];
    int i = p - 1;
    for (int j = p; j < r; j++) {
        if (A[j] <= pivot) {
            i++;
            swap(A, i, j);
        }
    }
    swap(A, i + 1, r);    // 피벗 이동
    return i + 1;        // 분할 지점 반환
}
```

```
void quickSort(int[] A, int p, int r) {
    if (p < r) {
        int q = partition(A, p, r);
        quickSort(A, p, q - 1);
        quickSort(A, q + 1, r);
    }
}
```

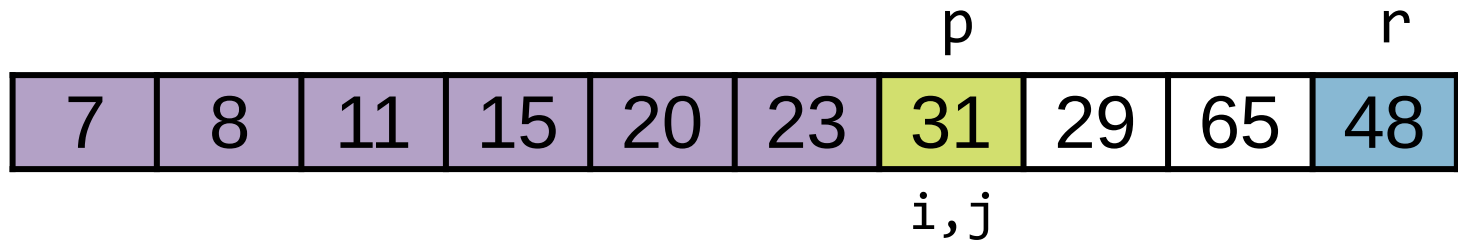
오른쪽 부분 배열 대상 정렬 (Pivot: 48)

						p			r
7	8	11	15	20	23	31	29	65	48
					i	j			

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

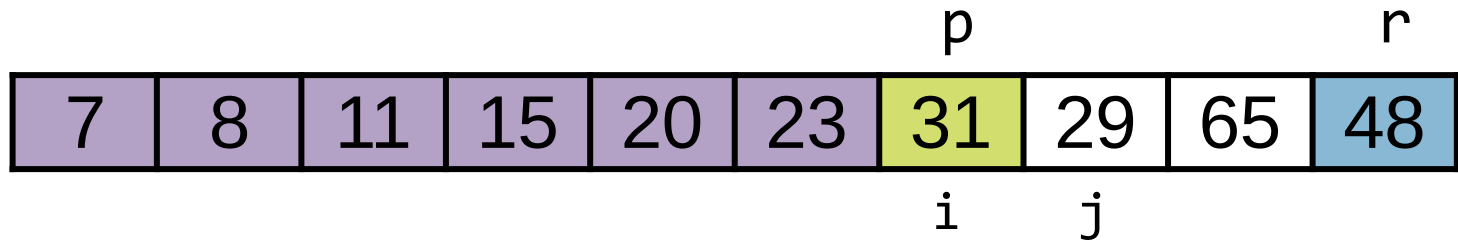
오른쪽 부분 배열 대상 정렬 (Pivot: 48)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

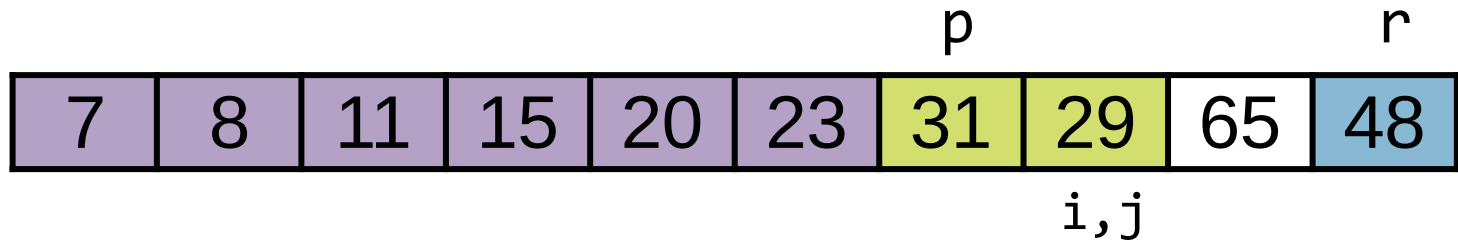
오른쪽 부분 배열 대상 정렬 (Pivot: 48)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

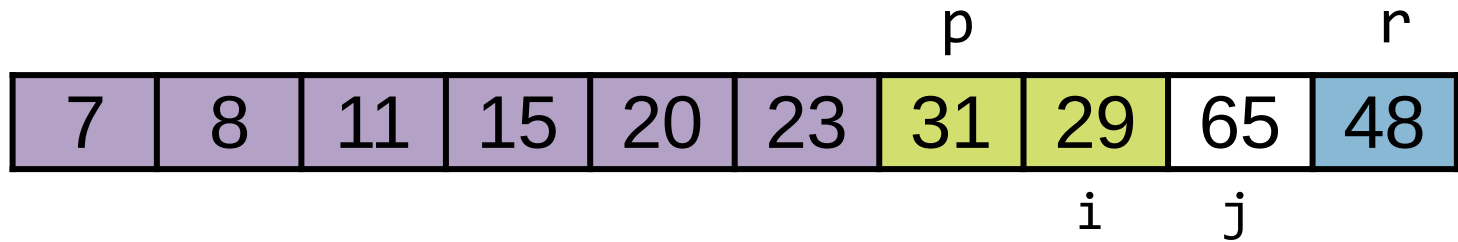
오른쪽 부분 배열 대상 정렬 (Pivot: 48)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

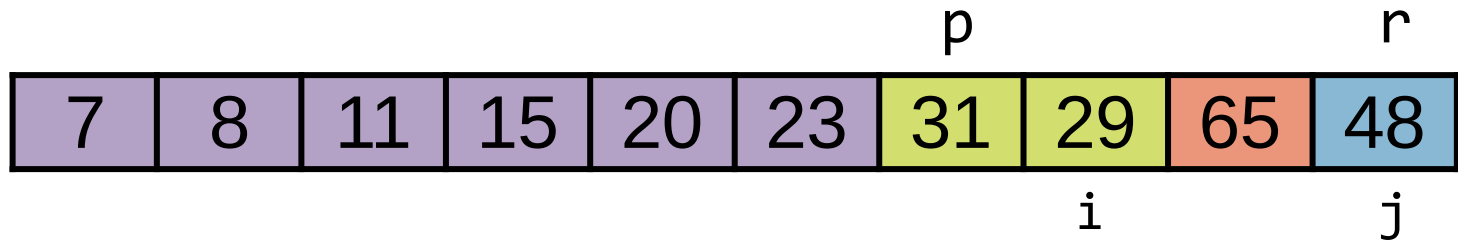
오른쪽 부분 배열 대상 정렬 (Pivot: 48)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬 (Pivot: 48)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

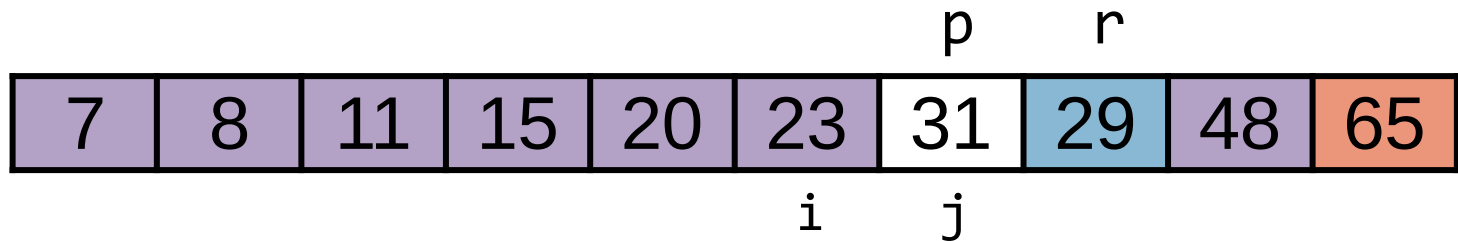
오른쪽 부분 배열 대상 정렬 (Pivot: 48)

						p		q	r
7	8	11	15	20	23	31	29	48	65
							i		j

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);  
    return i + 1;  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

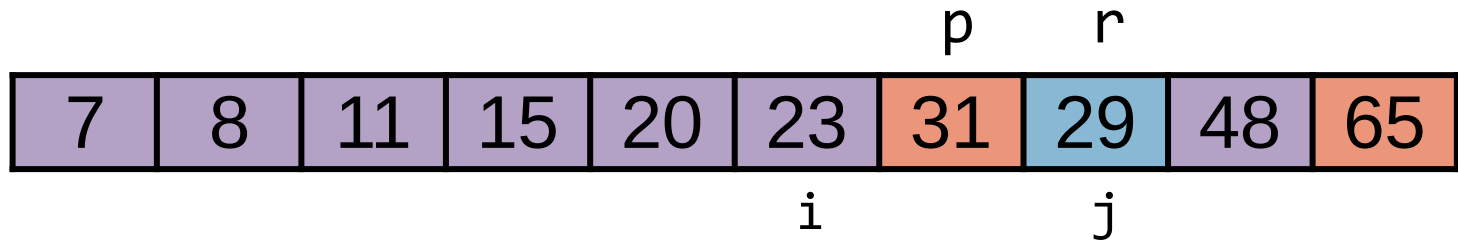
오른쪽 부분 배열 대상 정렬 (Pivot: 29)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

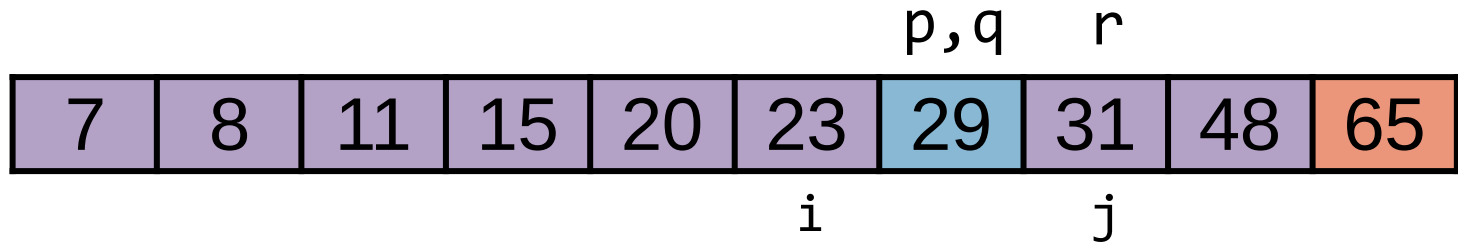
오른쪽 부분 배열 대상 정렬 (Pivot: 29)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬 (Pivot: 29)



```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r); // 피벗 이동  
    return i + 1;     // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬

					r	p			
7	8	11	15	20	23	29	31	48	65

```
int partition(int[] A, int p, int r) {  
    int pivot = A[r];  
    int i = p - 1;  
    for (int j = p; j < r; j++) {  
        if (A[j] <= pivot) {  
            i++;  
            swap(A, i, j);  
        }  
    }  
    swap(A, i + 1, r);    // 피벗 이동  
    return i + 1;        // 분할 지점 반환  
}
```

```
void quickSort(int[] A, int p, int r) {  
    if (p < r) {  
        int q = partition(A, p, r);  
        quickSort(A, p, q - 1);  
        quickSort(A, q + 1, r);  
    }  
}
```

오른쪽 부분 배열 대상 정렬

p, r (cf. p.63)

7	8	11	15	20	23	29	31	48	65
---	---	----	----	----	----	----	----	----	----

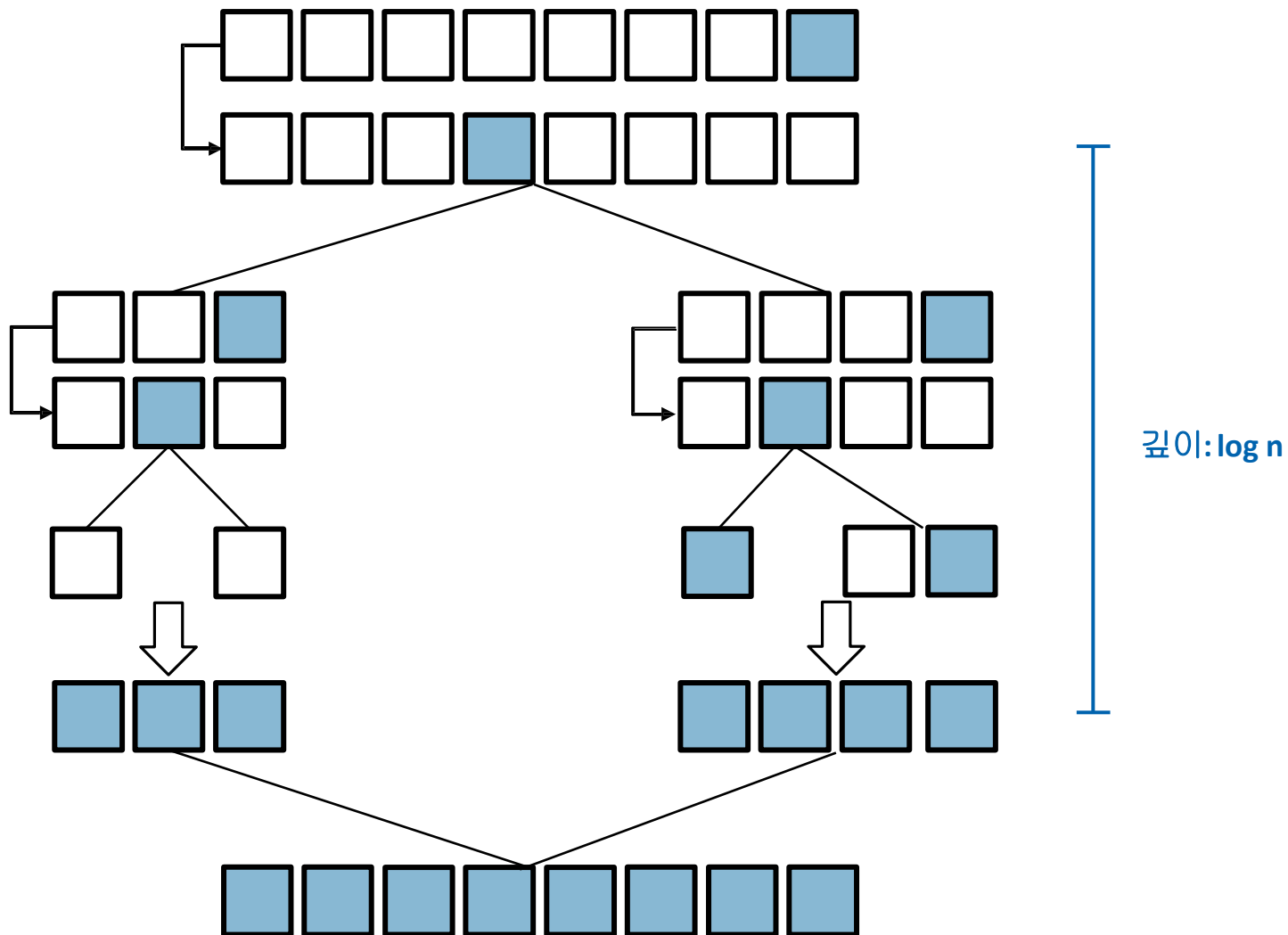
완성

```
int partition(int[] A, int p, int r) {
    int pivot = A[r];
    int i = p - 1;
    for (int j = p; j < r; j++) {
        if (A[j] <= pivot) {
            i++;
            swap(A, i, j);
        }
    }
    swap(A, i + 1, r);    // 피벗 이동
    return i + 1;        // 분할 지점 반환
}
```

```
void quickSort(int[] A, int p, int r) {
    if (p < r) {
        int q = partition(A, p, r);
        quickSort(A, p, q - 1);
        quickSort(A, q + 1, r);
    }
}
```

시간 복잡도

재귀의 깊이는 $\log n$ 이며, 각 레벨에서 총 n 개의 원소가 비교 연산 과정에 참여



Quick Sort vs. Merge Sort

- 평균 시간 복잡도는 서로 동일
- 메모리 공간 사용 측면에선 퀵 정렬이 더 유리

		Quick Sort	Merge Sort
시간	시간 복잡도 (평균)	$O(n \log n)$	$O(n \log n)$
	시간 복잡도 (최악)	$O(n^2)$ - 피벗이 매번 최소 또는 최대값으로 선택되는 경우 - 입력 배열이 이미 거의 정렬된 상태일 경우	$O(n \log n)$
공간	메모리 공간 복잡도	$O(\log n)$ - In-place 방식으로 수행 (기존 저장 공간을 재활용) - 재귀 호출로 인한 스택 공간 (트리의 깊이) 만을 필요	$O(n)$ - 병합 과정에서 크기 n의 보조 배열 필요 (e.g., merge의 result 배열)
	캐시 효율	높음 (연속 접근 가능)	낮음 (두 배열 왕복)

Look ahead

Priority Queue & Heap